

郑州轻工业大学

本科毕业设计（论文）

题 目 基于 Verus 的 reL4 权能空间的
形式化验证

学生姓名 苏爽

专业班级 计算机科学与技术（嵌入式软件）22-04

学 号 542213430420

学 院 计算机与人工智能学院

指导教师（职称） 张平原（讲师）

完成时间 2026 年 5 月 18 日

郑州轻工业大学
本科毕业设计（论文）任务书

题目 基于 Verus 的 reL4 权能空间的形式化验证

专业 计算机科学与技术(嵌入式软件) 学号 542213430420 名 苏爽

主要内容、基本要求、主要参考资料等：

一、主要内容

针对 Rust 微内核 reL4 中能力空间（CSpace）核心操作语义，开展基于 Verus 的形式化验证研究。以 reL4 的能力空间管理模块为对象，建立相应不变量体系与前后置条件规格，结合验证规模统计、构建结果及运行测试，对所提出方法的可行性、结论边界和改进方向进行分析总结。

二、基本要求

分析微内核能力系统与能力空间验证需求，明确研究对象、验证边界和总体技术路线，完成相关文献调研与方案论证。基于 Verus 建立能力空间分层抽象模型，给出能力关系、槽位状态、MDB 结构、CDT 派生关系及管理器等良构性条件的形式化定义。完成系统性实验与结果分析，包括验证规模、可信边界、运行时形态和测试结果等内容，总结当前工作的局限性并提出后续改进方向。

三、主要参考资料

- [1] Klein G, Elphinstone K, Heiser G, et al. seL4: Formal Verification of an OS Kernel[C]. Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles, 2009: 207-220.
- [2] 徐家乐, 王淑灵, 李黎明, 等. 操作系统内核权能访问控制的形式验证[J]. 软件学报, 2025, 36(8): 3570-3586.

完 成 期 限： 2026 年 5 月 18 日

指导教师签名： 张平原

专业负责人签名： 王华

2026 年 1 月 6 日

基于 Verus 的 reL4 权能空间的形式化验证

摘 要

微内核会把更多服务交给用户态组件来完成，所以它的结构边界相对更清楚，可信计算基也会更小一些，因此也更适合作为高可信论证的对象，对于 seL4/reL4 这类能力系统微内核来说，能力空间，也就是 Capability Space，简称 CSpace，并不是一个普通的表结构，它不只是管理能力槽位里面的内容，还要维护派生关系、路径解析，以及删除和回收等关键行为，如果这个模块出现错误，访问控制语义就可能会被直接破坏，基于这个考虑，本文选择 reL4 的 CSpace 模块作为研究对象，讨论怎样使用 Verus 来验证这一类局部改动会影响全局约束的核心操作。

本文在 seL4 能力语义和 Verus 系统验证方法的基础上，构建了一个由能力语义、槽位对象、MDB 图关系、能力派生树，以及管理器组合层组成的抽象模型，并设计了覆盖能力内容、前驱后继一致性、派生关系、badge 语义、untyped 子约束和 zombie 约束的不变量体系，同时采用分层后状态的方法，来组织核心操作的证明。

本文得到的主要结果，主要集中在 CSpace 管理器核心层，目前，路径解析、局部更新和删除传播这三条核心操作线，已经可以放在同一套分层证明框架下进行讨论，resolve_address_bits 所代表的解析路径，以及 cte_insert、cte_move、cte_swap 所代表的局部更新路径，已经形成了覆盖槽位语义、MDB 图后状态、CDT 派生状态和管理器层 wf 恢复的机械化结果，围绕 delete_one、delete_all 和 revoke 的删除传播路径，也已经形成了以阶段化契约、部分正确性和 wf 保持为中心的证明结果，全文还结合代码规模统计、14v 规模参照、运行时形态观察，以及 spike 平台上的验证和测试结果，对这些成果进行了评估，本文有意把结论边界收在管理器核心层之内，没有把公共封装接口、系统调用级行为和整内核端到端验证，一起计入已经得到的结论。

关键词 形式化验证；微内核；能力空间；Verus

Formal Verification of the reL4 Capability Space Based on Verus

ABSTRACT

Microkernels hand over more services to user-level components, so they usually have clearer structural boundaries and a smaller trusted computing base, and they are also more suitable as objects for high-assurance reasoning. For capability-system microkernels such as seL4/reL4, the Capability Space, or CSpace, is not an ordinary table structure. It not only manages the contents stored in capability slots, but also maintains key behaviors such as derivation relations, path resolution, deletion, and reclamation. Once an error appears in this module, the access-control semantics may be directly broken. Based on this consideration, this thesis selects the CSpace module of reL4 as the research object, and discusses how to use Verus to verify this kind of core operation where local changes may affect global constraints.

Based on the capability semantics of seL4 and the system verification method of Verus, this thesis builds an abstract model composed of capability semantics, slot objects, MDB graph relations, capability derivation trees, and the manager composition layer. It also designs an invariant system covering capability contents, predecessor-successor consistency, derivation relations, badge semantics, untyped child constraints, and zombie constraints. At the same time, a layered post-state method is adopted to organize the proofs of core operations.

The main results of this thesis are mainly concentrated on the core layer of the CSpace manager. At present, the three core operation lines of path resolution, local update, and deletion propagation can already be discussed under the same layered proof framework. The resolution path represented by `resolve_address_bits`, and the local update paths represented by `cte_insert`, `cte_move`, and `cte_swap`, have formed mechanized r

esults covering slot semantics, MDB graph post-state, CDT derivation state, and manager-level wf recovery. The deletion propagation path around `delete_one`, `delete_all`, and `revoke` has also formed proof results centered on staged contracts, partial correctness, and wf preservation. This thesis also evaluates these results by combining code-scale statistics, the l4v scale reference, runtime-shape observations, and verification and testing results on the spike platform. The conclusion boundary is deliberately limited to the core layer of the manager. Public wrapper interfaces, system-call-level behavior, and whole-kernel end-to-end verification are not included in the existing conclusions.

KEY WORDS Formal Verification; MicroKernel; Capability Space; Verus

目 录

摘 要	I
ABSTRACT	II
1 绪论	1
1.1 研究背景与研究意义	1
1.2 国内外研究现状	2
1.3 问题定义与研究难点	4
1.4 论文结构安排	5
2 背景与相关技术基础	6
2.1 微内核、seL4 与权能机制基础	6
2.2 l4v 与 seL4 形式化验证	6
2.3 Rust 验证现状与 Verus	7
2.4 Verus 的系统验证特点	8
3 研究对象、范围与关键问题	10
3.1 验证对象与模块职责	10
3.2 验证范围与对象分解总览	10
3.3 能力空间验证的关键问题	11
3.4 本文方案与研究边界	12
4 形式化方法设计	13
4.1 总体验证思路	13
4.2 分层抽象模型	14
4.3 管理器状态表示	15

4.4 不变量体系.....	15
4.5 Hoare 风格契约的证明义务分解.....	16
4.6 可信边界建模与桥接策略.....	18
5 验证实现.....	20
5.1 实现对象与组织方式.....	20
5.2 路径解析类操作：以 resolve_address_bits 为代表.....	21
5.3 局部更新操作验证实现.....	23
5.4 删除传播操作验证实现.....	25
6 验证结果、分析与讨论.....	27
6.1 评估目标、实验设置与指标.....	27
6.2 代码与证明规模统计.....	27
6.3 与 14v 的规模参照.....	28
6.4 与证明擦除相关的运行时形态分析.....	28
6.5 构建、验证与 sel4test 运行结果.....	29
6.6 结论边界与接口与系统级扩展.....	31
6.7 研究局限和后续展望.....	31
结束语.....	32
致 谢.....	33
参考文献.....	34

1 绪论

1.1 研究背景与研究意义

操作系统内核处在软件和硬件相互连接的核心位置，像进程管理、内存管理、设备访问，还有权限控制这些基础功能基本都需要内核直接参与完成，也是因为内核承担了这些比较底层的职责，所以它对系统稳定性和安全性的影响很直接，有时候即使只是比较小的实现错误也可能导致上层应用出现异常，情况严重的时候甚至会引起整个系统崩溃^[1]。软件测试、代码审查和静态分析等传统方法能够发现其中一部分缺陷，但是放在内核这种状态空间比较复杂、执行路径也比较多的系统里面来看，它们很难覆盖所有可能出现的运行情况，形式化验证在这里提供了一种更加严格的分析办法，它用数学化的方式描述系统应该满足的性质，再通过机器检查来证明实现和规约之间保持一致从而为内核正确性提供相对更强的论证基础。

围绕操作系统内核形式化验证，国内已有学者进行过较系统的总结。钱汉伟等从验证理论、语言和工具三个方面梳理了相关研究，并指出验证成本高、工具能力有限仍然是该方向推进中的重要制约因素^[2]；钱振江等则把已有工作概括为围绕形式化设计、规格描述和验证方法展开的持续积累^[3]。放到国际研究背景下看，seL4 团队的工作更直接地展示了微内核形式化验证的可行路径：SOSP 2009 的研究给出了从抽象规范到 C 实现的功能正确性证明^[4]，TOCS 2014 又继续报告了访问控制、二进制语义保持和时序分析等更完整的系统级结果^[5]。这些研究说明，微内核确实可以成为形式化验证的对象；但另一方面，完整内核的规模和复杂度也意味着验证成本很高，直接处理整个内核并不适合作为本文的切入点。

据此，本文将研究对象收缩到内核中的一个核心模块，也就是基于权能机制，也称 capability 的 CSpace，选择 CSpace 主要有两个原因，第一，它在内核中承担对象引用、权限控制和能力派生等关键职责，和访问控制语义之间有比较直接的关系，第二，和完整内核相比，它的功能边界相对清楚一些，更适合在 Rust/Verus 环境下进行建模和验证，在 seL4 中，权能不只是用来引用内核对象，还会带有相应的访问权限，并且会通过复制、移动、删除和派生等操作，影响系统中的权限分布^[1,7]，因此，CSpace 并不只是一个用来保存对象槽位的数据结构，它还需要在同一个结构中维护对象引用、权限约束和能力派生关系，访问控制能不能可靠，在很大程度上取决于这些关系能不能在 CSpace 操作过程中被正确保持。

从功能正确性的角度来看,对能力系统进行验证有比较明确的必要性,seL4 关于完整性和授权封闭性的研究说明,能力机制是高层安全语义能够成立的一个重要前提^[8],另外,已有研究还把证明推进到了编译后程序层面,并指出源代码层面的结论,不能自动推广到二进制实现^[9]。

随着系统软件实现语言的发展,Rust 因为在内存安全和底层控制能力之间取得了比较好的平衡,逐渐受到操作系统研究的关注,reL4 正是在这种背景下出现的,它延续了 seL4 的微内核设计思路,同时也把高可信系统实现带入 Rust 环境,不过,Rust 本身提供的安全保证主要还是集中在内存安全和类型安全方面,并不能自动解决功能正确性问题,特别是在标准库和底层库中大量使用 unsafe 的情况下,这个问题会更加明显^[10],Verus 的价值在于,它允许在同一套 Rust 环境中同时表达代码、规格和证明^[11,12],对于 CSpace 这类边界相对明确,同时又涉及权限语义的内核模块来说,Verus 提供了一个比较合适的验证基础。

所以,本文围绕 CSpace 构建 Verus 形式化建模与验证框架,主要是想尝试在 Rust/Verus 环境中表达能力槽、能力派生关系,以及权限检查过程的核心语义,具体来说,本文将 CSpace 的插入、删除、查找和派生等操作,抽象为 Verus 中的数据结构和不变式,并围绕这些关键操作开展验证,这样做一方面是在探索 seL4/l4v 已有验证经验怎样转化到 Rust/Verus 环境中,另一方面也希望为内核 Rust 重写过程中,从代码安全走向行为可信,提供一个更具体的实现路径和实践参考。

1.2 国内外研究现状

围绕可信内核和系统验证这一方向,现有文献大致可以归为五条研究线索。

第一条主线是以 seL4 为代表的微内核功能正确性与安全性质验证。SOSP 2009 的 seL4 工作把研究目标明确为抽象规范到 C 实现的功能正确性证明^[4],TOCS 2014 又把功能正确性、访问控制、二进制语义保持和系统初始化等结果并置到同一套系统级论证中^[5]。From L3 to seL4 对 L4 系列发展的回顾则强调,最小化、性能和可验证性并不是彼此孤立的追求,而是共同塑造内核边界的设计驱动^[6]。在这一基础上,seL4 Enforces Integrity 进一步把能力系统与完整性和授权封闭性联系起来,说明能力机制并不只是内核中的一种实现细节,而是安全性质得以成立的结构前提^[8]。中文综述文献对这一方向的归纳也指出,微内核和安全内核一直是操作系统形式化验证最集中的对象之

一^[2]。因此，现有研究实际上已经给出了一个比较清楚的共识：能力系统是值得做强语义验证的微内核核心部分。

第二条主线关注高层证明如何继续向下连接到更低层实现。Translation validation for a verified OS kernel 提示了一个重要问题：即使抽象规范与源代码之间的语义关系已经建立，编译、链接以及后续更底层的实现过程，仍然可能成为验证结论继续外的薄弱环节^[9]。与此相近，在综述中也都强调，验证工作需要说明其所处层次、适用边界以及依赖的工具条件^[2]。可见，文献中所谓已验证的表述，并不是一种不加区分的整体性判断，而是需要明确指出验证结论究竟停留在哪一个抽象层次上。

第三条主线涉及 Rust 系统验证与 Verus 工具生态。从语言基础来看，RustBelt 从形式语义角度讨论了 Rust 的安全主张、所有权类型系统，以及 unsafe 库实现所依赖的验证前提^[10]。这一工作说明，语言层面的安全承诺并不能自动推出具体模块的功能正确性。中文相关文献虽然多数尚未直接讨论 Verus，但也有工作已经将形式化验证方法概括为最有潜力确保操作系统安全可信的方法，并尝试按照单元、模块和系统三个层次来组织验证过程^[13]。与本文关系更直接的是 Verus 的代表性工作：这类研究把重点放在工具平台本身，即如何在同一套 Rust 语法环境中同时书写执行代码、规格和证明，并借助 SMT、ghost 状态和线性资源机制来表达和验证更复杂的程序性质^[11]。

第四条主线是 Verus 生态下的大规模系统化案例。Hance 的博士论文^[14]讨论了并发系统代码验证的组织方法；IronFleet^[15]虽然早于 Verus，但其“状态机精化 + 程序逻辑”的结合方式深刻影响了后来的系统验证实践。随后，IronSync^[16]、PoWER^[17]、Cort enMM^[18]、Atmosphere^[19]以及 VeriSMo^[21]分别在并发推理、崩溃一致性、内存管理、Rust 内核和安全模块等方向展示了统一验证平台的工程潜力。就本文的问题意识而言，这一系列工作最重要的启示不是某篇论文中的局部技巧，而是一个更稳定的判断：Verus 已经开始具备支撑中大规模系统子模块验证的现实条件^{[22][23]}。因此，在 Rust/Verus 生态中讨论 reL4 CSpace，不再只是方法上的试探，而是可以嵌入现有系统验证主线的研究选择。

第五条主线是与权能访问控制直接相关的验证工作。在中文研究中，徐家乐等将操作系统正确性理解为 API 函数具体实现与抽象规范之间的精化关系，并基于此对某微内核的权能访问控制模块进行了 Coq/CSL-R 验证^[1]。同一研究还特别指出，在权能访问控制的实现中，不仅所有任务的权能空间构成多个树结构，而且广泛存在访问、修改

以及递归删除等操作^[1]。对本文而言最直接的启发是：树结构维护、复杂嵌套数据结构、一致性不变式以及递归删除路径，确实构成了这一问题域的共性难点。

表 1-1 相关工作与本文定位比较

工作方向	主要对象	工具或框架	本文相对该方向的新增点
seL4/l4v 功能正确性与安全性质	微内核抽象规范、C 实现与安全性质	Isabelle/HOL	将其能力语义强度与边界口径迁移到 reL4 的 Rust/Verus 代码基
Verus 与 Rust 验证基础	Rust 语言基础与系统代码验证平台	RustBelt、Verus	把通用 Rust/Verus 验证能力收束到能力空间这一高语义密度内核子系统
Verus 系统案例	并发、存储、内存管理、Rust 内核	Verus 生态	给出面向 CSpace 的分层后状态组织、边界控制和评估证据组合方式
本文	reL4 sel4_cspace 管理器核心层	Rust + Verus	在真实 reL4 代码基中同时回答验证边界、证明组织与评估证据三类问题

综合前述文献可以看出，现有研究已经分别回答了若干关键问题：能力系统为何值得验证^[4]；验证结论应如何分层表述^[5]；Rust/Verus 是否能够支持系统代码的验证^[13]；以及在权能访问控制验证中通常会遇到哪些难点^[24]。然而，从已有中文文献来看，相关工作更多集中在综述分析、对象语义模型构建、需求层面或嵌入式系统验证方案等方向^[2]；而对于 reL4 这类 Rust 微内核实现，针对 CSpace 核心层的专门研究仍相对较少。本文的工作正是针对这一空缺展开：在不脱离现有 Rust 实现的前提下，将解析、局部更新以及删除传播等核心操作纳入同一套 Verus 证明框架。

1.3 问题定义与研究难点

基于上述缺口，本文关注的核心问题可以概括为：在 Rust/Verus 生态下，如何为 reL4 中能力空间的核心操作层建立一套既保持 seL4 语义强度、又适合当前代码实现形态的形式化验证框架。围绕这一目标，本文聚焦以下四个研究问题。

第一，如何为 reL4 CSpace 划定一个既足够强、又可逐步推进的验证边界？

第二，如何把能力语义、槽位语义、MDB 图语义、CDT 派生语义和管理器组合语义组织为统一的 Verus 分层模型？

第三，如何在同一证明框架中同时覆盖路径解析、局部更新和删除传播三类差异明显的核心操作？

第四，本文最后得到的结果究竟有多强，付出了多大实现与证明代价，运行时代码形态和系统兼容性又该怎样评估。

1.4 论文结构安排

全文共分为六章。第 1 章先介绍研究背景和相关工作并提出本文需要解决的问题。第 2 章主要介绍能力系统、seL4/reL4、RustBelt 与 Verus 等的基础内容。第 3 章开始进入本文的工作，对需要验证的对象、研究的范围和关键的问题作出界定。第 4 章说明本文采用的形式化方法设计，内容主要包括分层的抽象模型、不变量的体系和证明的组织方式。第 5 章开始围绕解析、更新和删除的三大类操作，介绍其验证和实现的过程。第 6 章集中给出机器验证的结果、规模的统计、运行时形态的观察、测试的结果，以及相应的结果分析、边界、局限和后续的展望。

2 背景与相关技术基础

2.1 微内核、seL4 与权能机制基础

微内核的基本思路是将必须在特权态执行的最小机制保留在内核中，而把更多策略和服务交给用户态组件处理，这样的内核设计有助于系统隔离、验证以及构建可信体系^{[2][3]}。从《From L3 to seL4》对 L4 系列经验的总结可以看出，这种设计长期遵循最小化原则，同时兼顾关键路径的性能^[6]。seL4 正是在这一思路下发展出来的典型能力系统微内核，它提供进程间通信、线程管理、虚拟内存和访问控制等最小核心机制，并在权能机制的基础上实现访问控制^{[5][7][8]}。

在这样的系统中，权能机制承担了对对象引用、访问控制和权限传播这几项作用。徐家乐等将权能访问控制描述为一种细粒度的访问控制机制，强调它把访问权限直接与对象和操作绑定^[1]；seL4 手册则是从系统接口角度来说明，必须持有足够访问权限的权能才能请求相应的内核服务^[7]。也就是说，线程是否能够访问某个对象取决于其是否持有相应能力。

seL4 中的 capability space 由 CNode 递归组织而成。手册对这一点的说明非常具体：capability 在概念上位于 capability space 的某个 slot 中，而 capability space 则由内核管理的 CNode 构成有向图结构；能力地址由路径上各级 CNode 的 slot index 串接而成^[7]。因此，CSpace 既是能力的存放结构，也是线程进行对象访问时的寻址结构。徐家乐等人的中文相关工作也从实现角度说明，权能空间并不是简单数组，而是由多个树结构和嵌套数据结构共同构成^[1]。

从基本操作看，seL4 手册把能力管理主要归入 CNode 方法之中，包括 Mint、Copy、Move、Mutate、Rotate、Delete、Revoke 和 SaveCaller 等^[7]。其中，Mint 和 Copy 用于从已有能力产生新能力，Move 和 Rotate 用于调整槽位中的能力位置，Delete 用于删除指定槽位中的能力，而 Revoke 则会递归删除由某个能力派生出的子能力^[7]。这些操作共同构成了能力系统最基本的管理接口，也决定了后续能力解析、派生维护和删除传播等问题会以什么形式出现。

2.2 l4v 与 seL4 形式化验证

l4v 与 seL4 的相关文献已经把能力空间放进一套较清楚的形式化验证分层之中。SOSP 2009 给出的是从抽象的规范到 C 实现的正确性证明^[4]；TOCS 2014 是在此基础上继续汇总访问控制、二进制语义保持、初始化和时序分析等结果^[5]。

就能力空间本身来说, seL4 手册和 l4v 抽象规范已经给出了比较完整的语义描述: 线程的 CSpace 可以理解为从根 CNode 开始可达的一部分有向图, 能力地址是由路径上各级 CNode 的 slot index 拼接而成, 解析的时候需要逐步消耗 guard 和 radix 所对应的位串^[7], 所以能力查找是一个沿 CNode 递归下降的路径解析过程。

l4v 中对应的抽象规范把这套语义进一步程序化。相关理论首先把 capability space 描述为从线程 CSpace 根开始、由多个 CNode 构成的有向图, 并在这一抽象上定义 capability 查找、复制、插入、移动、交换、删除、finalise_cap 与 cap_revoke 等核心操作。以 resolve_address_bits 为例, 抽象规范把它写成沿多个 CNode 递归下降的查找过程: 如果当前能力是 CNode 能力且位串尚未耗尽, 就继续按 guard 和 radix 规则向下解析; 否则返回当前槽位或相应异常情形。这样一来, 路径解析、能力转移和删除传播都不再只是实现代码怎么写, 而被提升成了抽象状态上的语义操作。

在这套抽象语义里, 能力派生树和删除回收又是另一条核心主线。SOSP 2009 在讨论能力销毁时已经说明, seL4 用 capability derivation tree 追踪对象能力的来源关系, 并通过 zombie capability 表达删除过程中的中间状态^[4]。手册则把 Revoke 解释为递归删除派生子能力, 把 Delete 解释为删除指定槽位中的能力^[7]。在 l4v 中, 这些内容进一步落实为 cap_revoke、delete、finalise_cap 和相关不变量证明。与 resolve_address_bits 一样, 它们共同构成了 CSpace 语义传统中最重要的几条操作线。

l4v 的不变量证明部分则继续说明这些抽象操作的保持性质, 例如 resolve_address_bits 的返回槽位满足相应存在性条件, 能力更新不会破坏抽象对象和 CNode 结构要求等。也就是说, 在 l4v/seL4 的形式化验证传统里, 能力空间一直不是单纯的函数库, 而是“抽象操作定义 + 不变量保持证明”同时存在的对象。后续在 Rust 环境中讨论 reL4 的能力空间验证时, 最直接的语义参照仍然是 seL4 手册和 l4v 已经建立起来的这一套框架。

2.3 Rust 验证现状与 Verus

Rust 之所以受到系统软件研究关注, 在于它同时提供低级控制能力和相对强的静态安全保障。所有权、借用、生命周期以及对数据竞争的静态约束, 使 Rust 在很多场景中比 C/C++ 更适合作为系统软件实现语言。然而这些机制主要保证的是内存与借用安全, 并不能自动导出某个算法满足抽象规范或某个状态机保持全局不变量之类更强的正确性的结论。

现有 Rust 验证研究大致可以分成两类。第一类是语言基础工作，以 RustBelt^[10]为代表，主要回答 Rust 的安全承诺在形式语义上依赖什么前提。RustBelt 明确指出，Rust 的所有权类型系统虽然提供了很强的安全保证，但标准库和底层库内部广泛使用 `unsafe` 特性，因此“Rust 是安全的”并不能被简单视作语言表面的直接结论，而需要进一步说明这些 `unsafe` 扩展在语义上为什么仍然是健全的^[10]。这一类工作主要讨论的是语言与库层面的 `soundness` 问题。

第二类则是面向程序的验证工作，重点是如何给现有 Rust 代码附加规格、组织证明以及处理共享状态、不变量和资源约束。国内近年的相关工作已经开始讨论如何把形式化方法落到真实 OS 工程中。例如王阳等把验证过程区分为单元、模块和系统三个层次^[13]。徐家乐等也说明当验证对象换成权能访问控制模块之后，复杂共享数据结构、树结构维护和递归删除会迅速成为主要难点^[1]。

因此，Rust 验证现状并没有停留在“语言本身更安全”这一层，而是逐渐形成了从语义基础到程序验证平台再到系统案例的连续研究路线。对后续工具平台的讨论，也正是在这一现状基础上展开的。

2.4 Verus 的系统验证特点

Verus 是面向 Rust 代码的 SMT 验证平台。OOPSLA 2023 的论文把它概括为一种在 Rust 语言内部同时书写执行代码、规格和证明的工具，并通过线性 `ghost type`、`borrow checking` 和 SMT 求解来支撑对资源、指针和并发相关性质的推理^[11]。与许多“程序一份、规格另一份”的工具不同，Verus 的基本思路是让规格、证明和可执行代码尽量共处于同一语言环境中：规格函数仍然写成 Rust 风格的函数，函数契约主要通过 `requires` 和 `ensures` 表达，证明则写成 `proof` 代码或引理函数附着在相邻实现附近。

从机制上看，Verus 中有几类组成部分对系统验证非常关键：第一是它会区分 `spec`、`proof` 和 `exec` 等不同模式，用来说明哪些代码参与运行、哪些代码只服务于证明，第二是它也支持使用 `ghost` 变量、`tracked` 资源和线性权限来表达这个值只在证明里存在，但必须被严格使用的对象，第三是它依赖 SMT 求解器自动处理大量一阶逻辑与代数条件，同时允许程序员通过引理、递归证明、`decreases` 子句和触发器选择来补充自动化不足的部分，四是 Verus 在验证完成后会擦除 `ghost` 代码与证明代码，因此这些部分不会进入最终机器代码^[11]。

SOSP 2024 和后续系统案例则说明, Verus 已经开始从“小型示例可证”走向“系统代码可证”^[12]。相关工作已经把它用于并发系统、崩溃一致性、内存管理、Rust 内核和安全模块等方向^{[12][16][17][18][19][20][21]}。这些案例并不直接给出能力空间验证的现成答案, 但它们至少表明, Verus 已经可以承载较复杂的数据结构、不变量和模块化证明组织

3 研究对象、范围与关键问题

3.1 验证对象与模块职责

本文的直接验证对象是 reL4 中负责权能空间模块。

从内核职责看，这个模块承担的事情要重得多：能力的创建、复制、移动、交换和删除都会经过这里，相应的槽位内容、链接关系和派生语义也都在这里被维护，更高层对象管理和系统调用路径又要依赖这里完成能力查找与地址解析。

对于整个 reL4 内核的验证来说，权能空间恰好在一个很适合验证的位置。往下走的话，会进入到位域表示和底层读写以及体系结构细节，而往上走的话，会直接碰到系统调用入口和更复杂的接口。而权能空间这一部分离抽象系统语义已经足够近，同时边界还没有扩散到难以控制的程度。

3.2 验证范围与对象分解总览

表 3-1 从层次，主要对象，职责三个方面确定研究范围。

表 3-1 验证层次、对象与结论口径总览

层次	主要对象	职责
能力语义层	capability 自身语义、对象/权限 /badge/untyped/zombie 信息	能力语义基础层
槽位对象层	槽位对象、槽位局部观察、局部方法	新架构下的重要承载层
MDB 图层	MDB 图结构、线性次序摘要、活跃槽位摘要	新架构下的独立结构证明层
CDT 派生层	派生树父子关系、原始性标记、派生状态转移	新架构下的独立派生语义层
管理器组合层	核心操作编排、总 wf 组合器	本文主体论证层次
对外接口层	公共封装接口、CNode 相关系统调用入口	与核心层相关，但不作为本文 主结论层

在操作类型上，本文仍将能力空间核心行为划分为四组。第一组是能力基础关系与查询操作，包括区域等价、对象等价、可回收性、最终能力判断等，它们为更复杂操作提供判定基础。第二组是查找与解析操作，以 `resolve_address_bits` 为代表，这一组决定能力路径如何被解释以及错误如何产生。第三组是局部更新操作族，包括 `cte_insert`、`insert_new_cap`、`cte_move` 和 `cte_swap`，它们代表能力空间中最典型的局部结构编辑。

第四组是删除与回收操作族，包括 `delete_one`、`delete_all`、`set_empty`、`reduce_zombie` 和 `revoke`，它们共同构成能力清理、传播删除和派生树收缩的复杂路径。

本文在后文出现 `cte_insert`、`insert_new_cap`、`cte_move` 和 `cte_swap` 时，若未特别标注，主要指 `CSpaceManager` 上的核心层方法。与之相连的 `kernel` 转发入口和兼容封装层在运行时上保持关联，但证明强度不同：本文主结论落在 `CSpaceManager::*`，不把外层封装自动视为已完成验证的对外接口。

图 3-2 将论文中的抽象层次与现有工程模块对应起来，使各层的位置、职责与结论口径可以直接追踪。

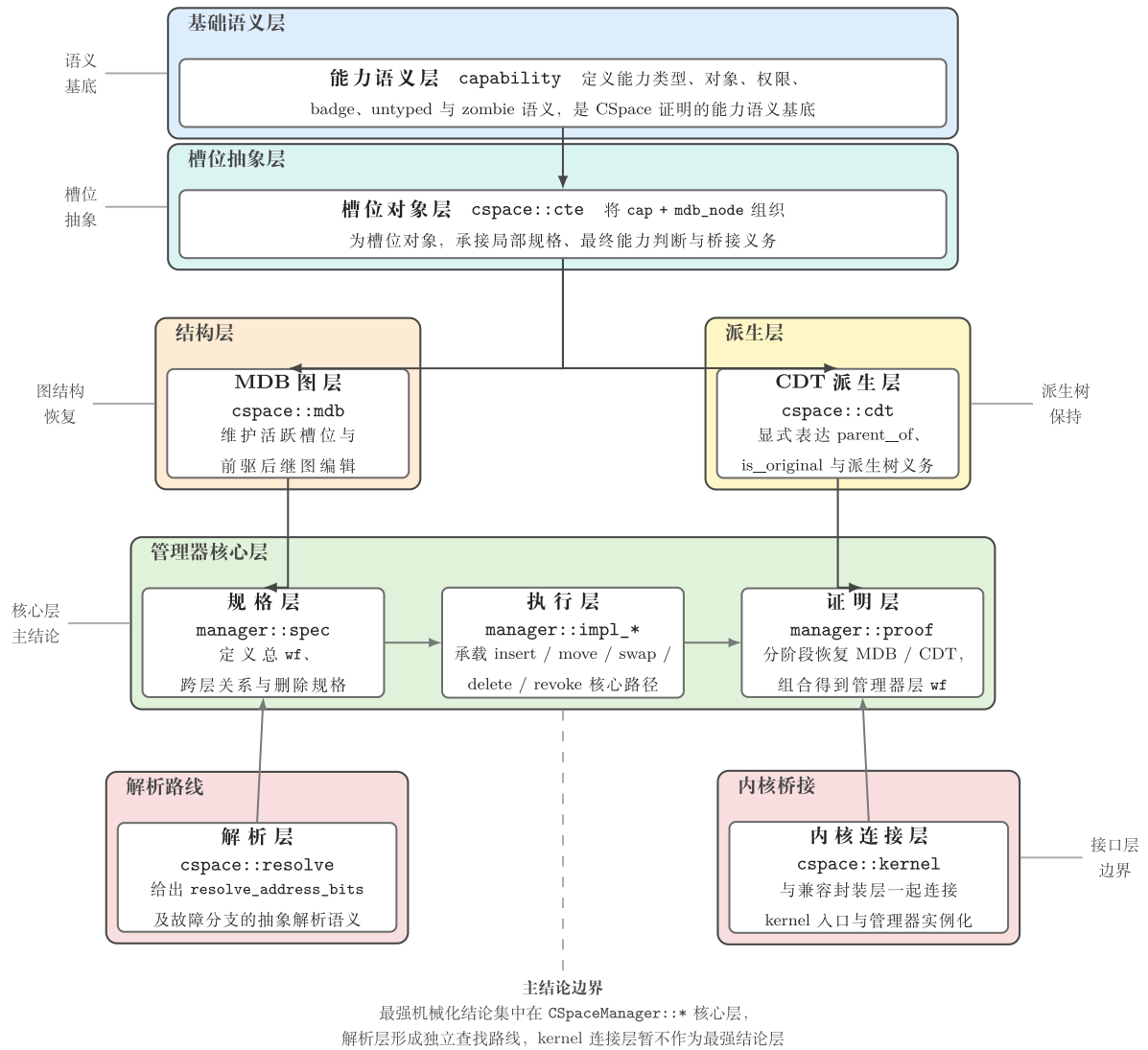


图 3-2 论文抽象层与现有工程模块对应关系

3.3 能力空间验证的关键问题

CSpace 相比普通数据结构更难验证，根本原因在于它同时承载对象语义、授权语义和拓扑语义。

能力内容本身就带有比较细的语义。不同种类的能力会指向不同对象，也会携带不同的权限位、`badge` 信息和对象相关参数；有些能力可以理解为同一区域的不同视图，有些能力在 `same_object_as` 下虽然等价，但在授权传播上并不完全相同。即在验证的时候要同时看能力有没有被复制到某个位置和判断在复制、移动或删除完成之后，区域、对象、`badge` 以及可撤销标记等语义是否仍然保持成立。

MDB 的前驱与后继关系、派生父子关系，以及原始/可撤销这类附加标记，都会让单个槽位的正确性无法脱离相邻槽位单独说明。一次 `cte_move` 表面上只是把能力从源槽位挪到目标槽位，但只要旧邻居、目标槽位原状态或来源父子关系处理得不够准确，整个派生链就可能被改乱。`cte_swap` 的情况更复杂，它相当于让两个局部子图同时发生更新。

更难处理的是删除路径。和插入、移动相比，删除相关操作 `delete_one` 要先完成空槽化，再进入 `finalise`、`zombie reduce` 和子节点处理；`delete_all` 与 `revoke` 还会把这种分阶段处理放进循环中，对一批相关槽位连续生效。这样一来，证明还要说明在阶段不断推进的过程中，哪些语义已经被回收，哪些部分还需要留给后续步骤处理。

最后还有一个很现实的问题：证明里使用的抽象状态，并不会天然等于运行时代码里的具体表示。异常值怎么映射，底层读到的局部信息怎样提升成抽象槽位视图，体系结构相关能力哪些需要纳入逻辑边界，这些地方如果处理得太粗，可信边界就会被放大；如果处理得太细，证明又会被大量表示细节拖住。本文后面专门区分桥接层和核心语义层，主要就是为了把这些缝隙控制在一个能说明、也能继续收缩的范围里。

3.4 本文方案与研究边界

具体来说，本文主要把 `resolve_address_bits`、`cte_insert`、`insert_new_cap`、`cte_move`、`cte_swap`、`delete_one`、`delete_all`、`set_empty`、`reduce_zombie` 和 `revoke` 作为核心分析对象。这些操作构成能力空间中典型的读、写、交换和删除行为，也是后文方法设计、验证实现和结果评估的主要承载对象。而面向内核的封装层、兼容入口，则不是本文的结论。

4 形式化方法设计

4.1 总体验证思路

reL4 CSpace 的复杂度是因为能力内容、图结构、派生关系和删除生命周期同时存在，如果把这些语义全部压到一层，证明结构会失去可复用性，局部更新、图恢复和派生树恢复之间的责任界线也会变得模糊。所以，本文采用分层验证架构。

该架构有四项设计原则。一是，语义和规约强度仍以 l4v 为准。二是，证明组织应该利用 Verus 在 Rust 环境中表达执行代码、规格与证明的能力。三是，验证对象收束于 CSpace 管理器层。第四是，复杂状态按语义归属拆分到槽位对象层、MDB 图层、CDT 层和管理器组合层。

表 4-1 本文 CSpace 验证架构层次总览

架构层次	主要语义责任	在论文中的作用
能力语义层	能力类型、对象、权限、badge、untyped 与 zombie 相关信息	语义基准
槽位对象层	单个槽位的能力内容、局部元数据和局部 观察	槽位局部后状态说明
MDB 图层	前驱后继、活跃槽位、局部图编辑与结构 恢复	拓扑结构恢复和图良构性
CDT 派生树层	父子派生关系、原始性标记和派生状态转 移	能力派生树语义
管理器组合层	操作编排、跨层一致性和总 wf 恢复	本文核心层主结论边界

表 4-1 和图 4-1 展示了本文采用的 CSpace 分层验证结构。在图中的下层负责去解释具体的能力和槽位的关系，中间层则是分别处理 MDB 图结构与 CDT 派生关系，最上层的管理器层则把这些局部结论组合为能力空间的整体结论。

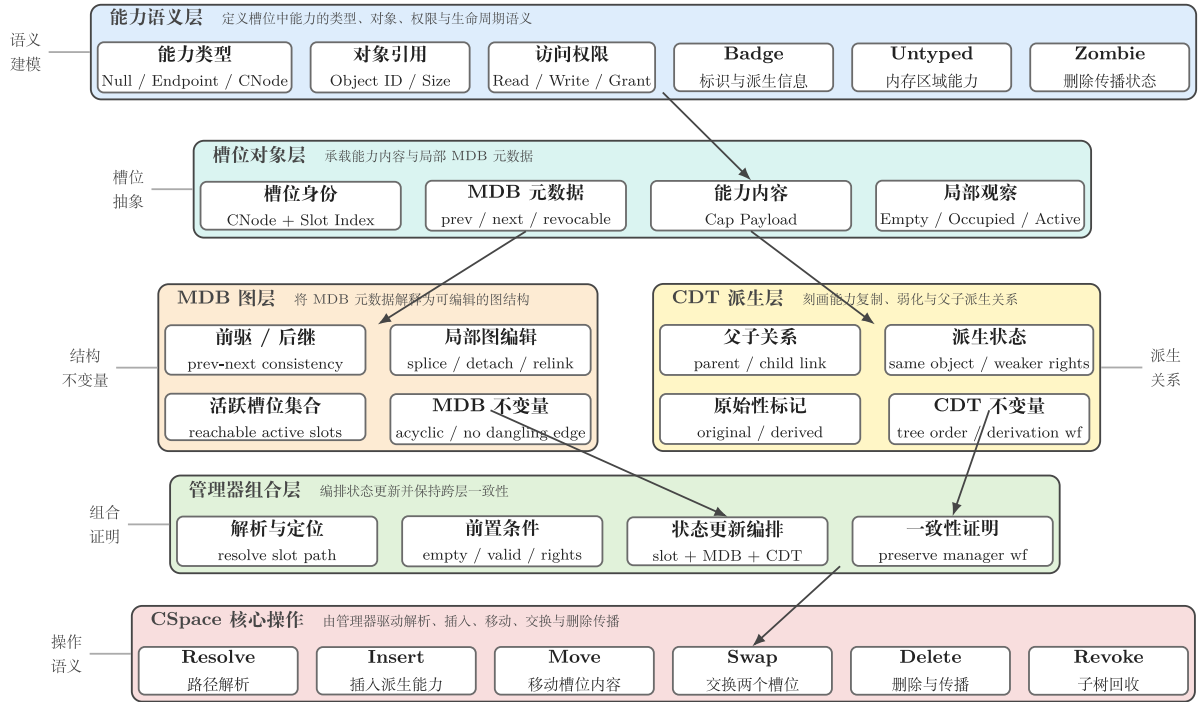


图 4-1 CSpace 分层验证结构

4.2 分层抽象模型

本文使用分层抽象的思路，将权能空间划分为语义层、链接节点层、槽位对象层、MDB 层、CDT 层和管理器层。

在权能抽象层中，本文保留对语义有决定作用的信息，如能力种类、对象标识、region、权限、badge、CNode 参数和 untyped 相关字段等，选择作为可信基而不展开所有机器级表示。这样做的目的是把能力在系统语义上代表什么从具体位布局中分离出来。

在链接节点层与槽位对象层中，本文把权能的内容与前驱、后继、可撤销性和首个带 badge 标记这些元数据给组合起来，形成可直接用于推理的槽位对象抽象，槽位对象层的语义去承担局部观察、派生、子节点检查和最终能力判断等槽位局部方法，也是分层验证架构中证明责任的关键层。

在 MDB 层与 CDT 层中呢，MDB 层是负责解释能力槽位之间的前驱后继关系、线性遍历次序、活跃槽位集合以及局部修改后的图结构恢复。CDT 层是负责解释能力派生树中的父子关系、原始性标记以及操作后的派生状态变化。

全局管理器抽象层负责的主要是组合：底层运行时执行先在各语义层建立后状态，再由管理器层恢复总的权能空间良构性。分层语义与整体正确性之间的关系，也集中在这一层表达。

4.3 管理器状态表示

能力空间管理器的核心作用，是为能力空间建立一个便于说明“整体正确性”的视角。每个槽位的能力内容与链接信息、派生关系和邻接修补，最终都在管理器层的状态与谓词中重新汇合。

统一状态表示会带来两点比较直接的收益，一是它可以避免为每个操作单独设计一套局部语义，不管是 `resolve_address_bits` 的路径解析、`cte_move` 的槽位编辑，还是 `revoke` 的删除传播，都可以放在同一个状态空间中讨论。二是它也让证明目标变得更清楚：每次操作都从一个满足若干前提和 `wf` 的管理器状态出发，执行后先得到若干分层局部后状态，再由管理器组合恢复总 `wf`。

4.4 不变量体系

本文把不变量的体系拆分成了若干可以分别讨论、最后再组合起来的约束，记作 `wf` 的总条件，其中包括槽位权限、槽位域基础条件、MDB 约束、CDT 结约束、空槽与活跃槽位对应、语义边约束、跨层一致性约束以及非 MDB 帧条件等几部分共同成立。

先看最基础的一类约束，也就是空槽和非空槽如何区分，空槽要求链接信息和附加的状态彼此是相容的，非空槽则需要和具体能力类型对应起来。插入、移动和删除就是依靠这两个条件来判断目标位置可写和当前位置应清理。

另一组关键约束则是 MDB 链接的一致性，前驱与后继关系决定了派生相关槽位如何被遍历、如何被修补，所以它们必须满足基本的互指一致、边界合理以及局部邻接关系正确等条件，本文把这些放到 MDB 层了。

与 MDB 链接一致性并行的，是父子派生与权限传播相关不变量。插入、复制和 `revoke` 类操作也会改变能力之间的派生语义，所以，验证过程中必须明确说明哪些槽位在操作后形成了新的父子关系，哪些槽位保持原始标记，而又有哪些 `badge` 或可撤销状态需要被设置或继续保留，本文将这一部分放入 CDT 层。

还有一组约束来自 `badge`、`untyped` 和 `zombie`。本文把 `badge` 与 `untyped` 通过语义边和跨层条件纳入管理器组合层，再把 `zombie` 相关的删除收尾语义单独放进核心层验证框架中。

在本文的工程组织中，这些约束最终会通过统一的 `wf` 谓词汇合起来。这里的 `wf` 并是分层结果重新闭合时的组合入口：MDB 层负责链接结构，CDT 层负责派生关系，其余跨层一致性和帧保持条件则补足层与层之间的连接，这样在证明管理器层的具体操

作的时候证明过程可以先分别恢复各层局部性质再把这些局部结论收束为新的 wf，而不用再在每个操作上反复重建整套全局语义。

4.5 Hoare 风格契约的证明义务分解

Verus 的函数验证可以看作 Hoare 逻辑在 Rust 程序验证中的工程化表达。对于一段程序 C ，Hoare 三元组通常写作：

$$\{P\}C\{Q\} \quad (4-1)$$

其中 P 是执行前必须满足的前置条件， Q 是执行后必须满足的后置条件。在 Verus 中，这一结构主要通过 `requires` 和 `ensures` 表达，对于包含循环的程序，Verus 通过循环不变式表达 Hoare 逻辑中的不变式规则；对于 SMT 自动化难以直接完成的证明，Verus 允许使用 `proof` 函数、`assert`、`assert_by`、`ghost` 状态和 `tracked` 权限等机制显式组织证明步骤。因此，本文并不重新定义新的程序逻辑，而是在 Verus 已有的 Hoare 风格契约、循环不变式和证明函数机制之上，组织 CSpace 的分层证明义务。

设 M 表示操作前的抽象能力空间状态， M' 表示操作后的状态， op 表示某个能力空间操作。本文将 CSpace 更新类操作的总体契约抽象为：

$$\begin{array}{c} \{wf(M) \wedge pre_op(M)\} \\ op \\ \{wf(M') \wedge post_op(M')\} \end{array} \quad (4-2)$$

其中 $wf(M)$ 表示操作前能力空间满足全局良构性， $pre_op(M)$ 表示该操作自身的调用前提， $post_op(M, M')$ 描述操作后状态与操作前状态之间的关系。对于普通算法验证来说， $post_op$ 往往只需要描述返回值或少量字段变化；但对于 CSpace， $post_op$ 必须同时说明槽位内容、MDB 图结构、CDT 派生树以及跨层语义的一致性。因此，本文进一步将 $post_op$ 分解为以下几类证明义务：

$$\begin{aligned} post_op(M, M') = & slot_post(op\ M, M') \\ & \wedge mdb_post(op, M, M') \\ & \wedge cdt_post(op, M, M') \\ & \wedge cross_layer_os(M') \\ & \wedge frame_ok(op, M, M') \end{aligned} \quad (4-3)$$

表 4-2 CSpace 证明义务与 Verus 机制对应关系

证明义务	对应 Verus 机制	CSPACE 中的含义
槽位后状态条件	ensures、规格函数、proof lemma	受影响槽位中的能力内容、空槽状态和局部元数据如何变化
MDB 后状态条件	ghost/tracked 状态、图结构规格、proof lemma	MDB 前驱后继、局部邻域修补、活跃槽位和图良构性恢复
CDT 后状态条件	ghost 状态、派生树规格、proof lemma	父子派生关系、原始性标记和删除传播后的派生树状态
跨层一致性条件	管理器层 ensures 与组合 lemma	槽位内容、MDB 边、CDT 父子关系、badge/untyped/zombie 约束之间的一致性
帧保持条件	不变性条件、有限变化邻域、循环不变式	未被操作影响的槽位、边和派生关系保持不变

下图进一步说明了这些证明义务之间的组合关系。

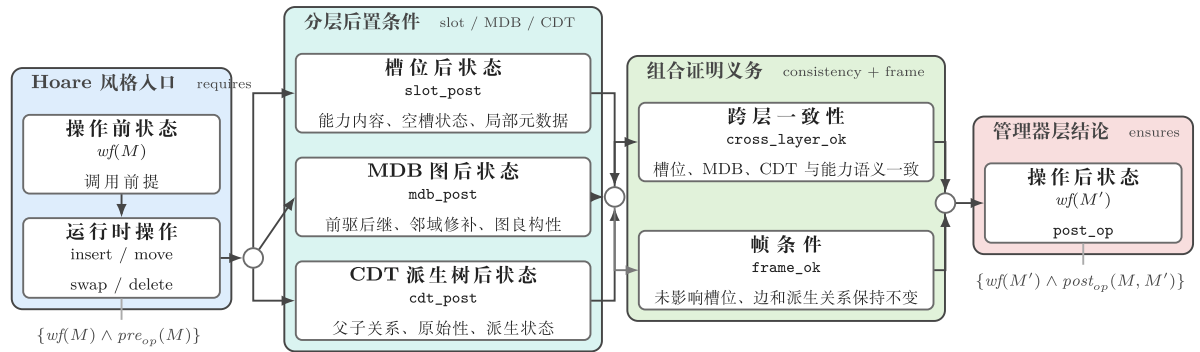


图 4-2 CSpace 更新类操作证明义务分解

上式是对 Verus 规格写法的论文级抽象，本文并未额外发明一套形式逻辑。对应到 Verus 中， $wf(M)$ 和 $pre_op(M)$ 通常出现在 requires 中， $wf(M')$ 和各类后状态关系通常出现在 ensures 中，slot_post、mdb_post、cdt_post 等则由不同语义层的规格函数和 lemma 承担。本文的证明表达仍然落在 Verus 提供的前置条件、后置条件、循环不变式和 proof 函数机制之内；论文中的分解式只是把这些工程化证明义务整理为更容易说明的研究结构。

当前工程中的 insert 最能体现前面所说的分层形态，操作完成后，状态会先落到槽位局部语义，进入 MDB 图结构后状态和 CDT 派生树后状态，最后由管理器层组合出

新的 wf.move 和 swap 在语义上也属于同一类局部更新问题，只是 swap 涉及两个非空槽位同时互换，证明时需要额外处理更多点对点映射和相邻关系变化。

查找类操作的证明义务形态有所不同，拿 resolve_address_bits 来说，这个操作的核心 Hoare 契约关注的是返回结果是否符合抽象查找语义。由于 resolve_address_bits 内部包含循环，Verus 还需要用循环不变式说明每次迭代时已解析前缀、当前能力、剩余位数和抽象路径语义之间的对应关系。

删除和 revoke 类操作的证明义务更混合一些，既有更新类特征，也有循环类特征。它们一方面要像局部更新操作一样说明槽位空槽化、MDB 修补和 CDT 收缩，另一方面又要像循环操作一样说明每一步删除传播都能保持阶段性不变式。其中 delete_post 还进一步包含空槽化结果、zombie 约简结果、子树收缩结果以及未处理区域的帧保持条件。

4.6 可信边界建模与桥接策略

任何现实系统验证工作都必须处理可信边界问题。

边界建模主要包括四种：第一是少量底层运行时观察与值构造例如异常状态映射、地址或槽位相关视图提取；第二是槽位局部语义中的少量底层辅助；三是管理器组合层中的剩余桥接；最后则是体系结构相关或删除收尾相关细节。

为了避免可信边界存在，但边界究竟在哪里并不清楚的问题，表 4-3 对本文保留的主要边界作出归纳。

表 4-3 可信边界类别与当前处理方式

边界类别	典型内容	当前处理方式
运行时观察边界	少量槽位视图提取、异常状态读取、值构造	作为桥接层输入进入抽象逻辑
槽位局部边界	派生、子节点检查、最终能力判定等局部语义结点	以局部契约显式保留
管理器组合边界	删除侧桥接、跨层组合、少量局部语义桥接	以分层后状态或局部契约形式保留

如果从方法来源上看，本文一头连着 seL4/l4v 的传统，一头连着 Verus 的系统验证实践。前者给本文提供的是语义强度上的参照，哪些能力关系要保住、查找和删除语义要精确到什么程度、不变量该怎样分类，这些都不能随意放松；后者给本文提供的则

是工程组织方式，即如何围绕管理器状态、分层后状态、帧条件和直接执行式证明来落地这些要求。

表 4-4 列出本文使用 l4v 的主要校准点。

表 4-4 l4v 语义校准点与本文对应关系

校准对象	l4v 侧主要作用	本文对应规格或证明位置
查找与解析	CNode 查找、guard/radix、lookup fault 情形	路径解析规格、分支证明与循环不变式设计
capability 关系	校准对象等价、区域等价、可撤销性、badge 等能力关系	能力关系规格与管理器公共语义条件
插入、移动与交换	校准槽位写入、MDB 邻接修补、派生关系与原始性状态	局部更新操作的执行、后状态与组合证明
删除与撤销	删除传播、finalise、zombie 和子派生链处理语义	删除传播、收尾语义与阶段化契约证明

5 验证实现

5.1 实现对象与组织方式

本章主要围绕三类比较有代表性的路径来说明验证的实现，分别是 `resolve_address_bits`、`insert` 和删除核心路径，其中 `resolve_address_bits` 主要对应路径解析，还有循环不变式，`insert` 对应局部更新操作里面的分层证明，删除核心路径则对应阶段化契约，以及生命周期收尾语义，这三类案例能够代表本文讨论的主要操作，是因为 `move` 和 `swap` 基本上可以复用 `insert` 所体现出来的局部更新证明思路，而删除路径又在这个基础上加入了阶段划分和循环推进，`resolve_address_bits` 则补充了只读循环和故障分支对应这一类不同的证明任务。

图 5-1 给出了 CSpace 验证实现的目录结构。路径解析、局部更新和删除传播相关实现已经在 `sel4_cspace` 仓库中形成明确模块划分。

```

root@rel4_dev_env:/workspace/rel4_kernel# tree sel4_cspace/src/ -d
sel4_cspace/src/
|-- arch
|   |-- aarch64
|   |-- riscv64
|-- capability
|-- cspace
|   |-- cdt
|   |-- cte
|   |-- manager
|   |   |-- impl_delete
|   |   |-- proof
|   |   |-- spec
|   |-- mdb
|   |   |-- table
|   |-- resolve
|-- deps
|-- kernel_api

```

16 directories
root@rel4_dev_env:/workspace/rel4_kernel#

图 5-1 CSpace 验证实现目录与核心文件截图

表 5-1 概括核心操作与旧版参考实现之间的语义连续性。

表 5-1 核心操作语义对照摘要

操作	在本文中的角色	与旧版实现保持一致的关键意图
<code>cte_insert</code>	局部更新代表案例	空目标槽检查、目标槽写入、插入到源槽之后、修补相邻 MDB 关系

insert_new_cap	插入线补充操作	读取 parent next、写入新能力、维护 parent/next 邻接关系
cte_move	带源槽清空的局部更新	目标槽继承源信息、源槽清空、邻居重定向到目标槽
cte_swap	双槽互换更新	双槽能力互换、双方邻居分别改指向新位置
delete_all/delete_one/revoke	生命周期与传播删除路径	finalise 后空槽化、删除槽时修补邻接关系、zombie 约简、对子派生链反复删除

表 5-2 进一步概括各操作在当前实现中的覆盖范围及其适用边界。

表 5-2 核心操作结论层级与适用边界

操作族	可支撑的最强结论	主要依赖语义层
resolve_address_bits	成功、提前停止与故障分支均已和抽象解析规格建立对应关系	resolve、capability、管理器层
cte_insert / insert_new_cap	已形成操作后关系与管理器层 wf 保持	cte_t、mdb、cdt、管理器层
cte_move / cte_swap	已形成局部更新后的管理器层 wf 保持	cte_t、mdb、cdt、管理器层
delete_one / set_empty	已形成单步删除后的安全性与管理器层 wf 保持	管理器层、mdb remove、删除契约
delete_all / reduce_zombie / revoke	已形成阶段化契约、安全性与管理器层 wf 保持的机械化结果	管理器删除传播层、cdt/mdb 协同

5.2 路径解析类操作：以 resolve_address_bits 为代表

resolve_address_bits 是本文中比较能代表查找这一类操作的例子，它的作用是沿着能力所指向的 CNode 结构，对地址位串进行解析，然后再按照 guard、radix 和剩余位数，判断下一步是继续向下查找，还是成功返回，或者产生故障，对于这一路径的验证，重点并不只是证明循环能够安全结束，而是还要说明执行中的循环过程，可以和抽象层面的解析含义保持一致，因此，本文围绕这个操作建立了比较明确的规约、循环不变式、终止性限制，以及成功分支和故障分支之间的对应关系，循环不变式一方面描述当前索

引、剩余位数和中间能力状态，另一方面也说明当前运行时位置，与抽象解析前缀之间的联系。

在此基础上，`resolve_address_bits` 已经承载了强于普通安全条件的语义证明。围绕它组织的一组 `refinement lemma` 分别对应 `guard` 过深、`guard` 不匹配、`level` 无效、首轮下降条件、当前节点到根节点语义提升、精确成功返回和提前停止返回等关键情形，因此 `resolve` 线的目标并不只是“函数能够返回”，而是不同分支上的返回值都与抽象解析规格一致。

下文表 5-3 总结 `resolve_address_bits` 的主要证明义务。

表 5-3 `resolve_address_bits` 证明义务摘要

证明义务	代表性代码产物	说明
初始根能力合法性	<code>resolve_invalid_root_case</code> 、 <code>resolve_invalid_root_result</code>	非 <code>CNode</code> 根能力进入故障分支
<code>guard</code> 过深	<code>lemma_root_guard_too_deep_contradiction</code> 、 <code>resolve_guard_too_deep_result</code>	保证 <code>guard size</code> 与剩余位数关系被正确处理
<code>guard</code> 不匹配	<code>lemma_root_guard_mismatch_contradiction</code> 、 <code>resolve_guard_mismatch_result</code>	保证查找失败被映射为 <code>lookup fault</code>
<code>level/radix</code> 无效	<code>lemma_root_level_invalid_contradiction</code> 、 <code>resolve_level_invalid_result</code>	保证 <code>radix</code> 与剩余位数不合法时不会产生伪成功
精确成功	<code>lemma_exact_success_current_refines</code>	返回槽位和剩余 <code>bits</code> 与抽象解析结果一致
提前停止	<code>lemma_early_stop_current_refines</code>	非精确但合法停止时保持抽象语义一致
当前节点到根节点提升	<code>lemma_current_refinement_lifts_to_root</code>	将循环中间状态的正确性提升为根调用的最终正确性

这些义务共同形成一条清晰的证明链：先通过根能力合法性、`guard` 匹配和 `level/radix` 合法性等 `lemma` 排除伪成功路径，再在循环中维持“当前节点状态与抽象解析前缀一致”的不变式，随后用 `lemma_exact_success_current_refines` 或 `lemma_early_stop_current_refines` 得到当前节点层面的结果，最后由 `lemma_current_refinement_lifts_to_ro`

ot 把中间结果提升为根调用的整体结论。因而，`resolve_address_bits` 同时覆盖了输入合法性、循环推进和返回语义三个层面，也说明本文方法不仅适用于局部槽位编辑，还能够处理非平凡控制流与故障语义。

图 5-2 给出了 `resolve_address_bits` 的实现位置及其对应证明条目。

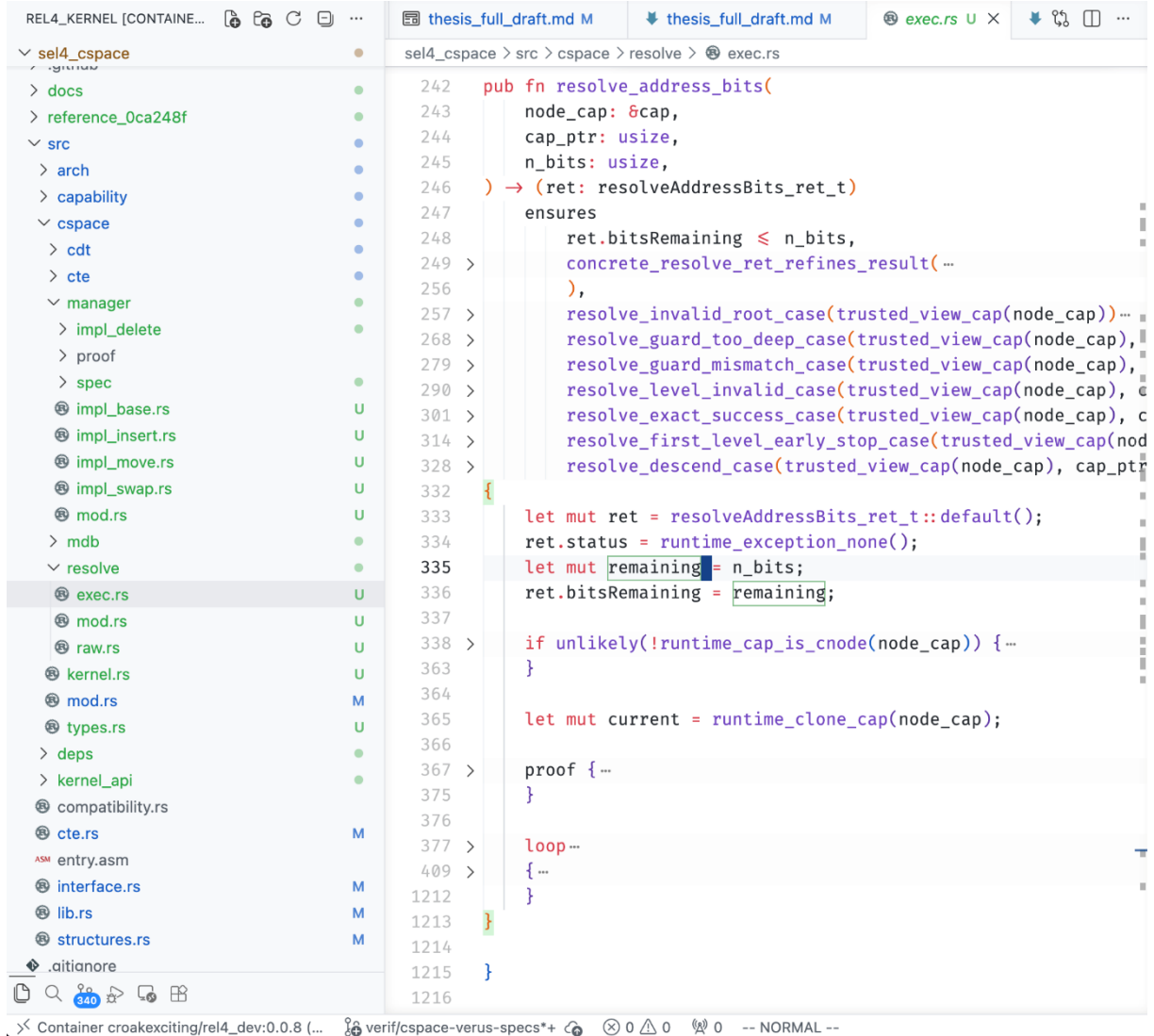


图 5-2 `resolve_address_bits` 实现与证明对应截图

5.3 局部更新操作验证实现

除了 `resolve_address_bits` 这个比较有代表性的查找案例之外，局部更新操作在验证实现时还要依靠一个更基础的事实，也就是 `cte_t`、`mdb` 和 `cdt` 这三层，其中 `cte_t` 主要承担槽位对象层的语义，`mdb` 负责图结构的恢复，`cdt` 则负责派生树状态的转移，所以局部状态恢复、图结构恢复、派生关系恢复和总 `wf` 恢复之间，就形成了比较清楚的分工，这也是局部更新路径和删除路径能够使用同一套证明骨架的前提。

从实现结构来看，本文没有把三类核心操作完全分开处理，而是通过一个统一的分层框架，来承接不同类型的操作，图 5-3 展示了局部更新路径中，实文件和证明文件之间的对应关系。

```

pub fn cte_move(&mut self, new_cap: &cap, src_slot: SlotPtr, dest_slot: SlotPtr)
  requires
    old(self).wf(),
    old(self).slot_dom().contains(src_slot),
    old(self).slot_dom().contains(dest_slot),
    !old(self).slot_is_empty(src_slot),
    old(self).slot_is_empty(dest_slot),
    old(self).get_cap(src_slot) == trusted_view_cap(new_cap),
    src_slot != dest_slot,
  ensures
    self.wf(),
    old(self).cte_move_rel(self, src_slot, dest_slot, trusted_view_cap(new_cap)),
  { ...
}

--
19 impl CSpaceManager {
20   pub fn cte_insert(&mut self, new_cap: &cap, src_slot: SlotPtr, dest_slot: SlotPtr)
21     requires
22       old(self).wf(),
23       old(self).slot_dom().contains(src_slot),
24       old(self).slot_dom().contains(dest_slot),
25       old(self).slot_is_empty(dest_slot),
26       !old(self).slot_is_empty(src_slot),
27       src_slot != dest_slot,
28       old(self).cte_insert_semantic_admissible(src_slot, dest_slot, trusted_view_cap(new_cap)),
29     ensures
30       self.wf(),
31       old(self).cte_insert_rel(...
32     ),
33   { ...
34 }

172
173
174   pub fn insert_new_cap(&mut self, parent: SlotPtr, slot: SlotPtr, capability: &cap)
175     requires
176       old(self).wf(),
177       old(self).slot_dom().contains(parent),
178       old(self).slot_dom().contains(slot),
179       old(self).slot_is_empty(slot),
180       !old(self).slot_is_empty(parent),
181       parent != slot,
182       old(self).insert_new_cap_semantic_admissible(parent, slot, trusted_view_cap(capability)),
183     ensures
184       self.wf(),
185       old(self).insert_new_cap_rel(
186         self,
187         parent,
188         slot,
189         trusted_view_cap(capability),
190       ),
191   { ...
275 }
276

```

```
impl CSpaceManager {
  pub fn cte_swap(&mut self, cap1: &cap, slot1: SlotPtr, cap2: &cap, slot2: SlotPtr)
    requires
      old(self).wf(),
      old(self).slot_dom().contains(slot1),
      old(self).slot_dom().contains(slot2),
      !old(self).slot_is_empty(slot1),
      !old(self).slot_is_empty(slot2),
      old(self).get_cap(slot1) == trusted_view_cap(cap1),
      old(self).get_cap(slot2) == trusted_view_cap(cap2),
      slot1 != slot2,
    ensures
      self.wf(),
      old(self).cte_swap_rel(
        self,
        slot1,
        trusted_view_cap(cap1),
        slot2,
        trusted_view_cap(cap2),
      ),
  { ...
  }
}

} // verus!
```

图 5-3 局部更新实现与证明文件对应截图

在局部更新方面，插入、移动和交换这几个操作已经形成了管理器层 wf 保持的证明结果，其中 insert 更能表现出分层结构的特点，执行入口会先完成目标槽的写入，还有 MDB 邻接关系的修补，并且在抽象层面上把这些内容汇总成 cte_insert_rel 这一类新旧状态关系，之后槽位层负责说明目标槽中的新能力内容，MDB 层负责说明新节点插入之后的前驱和后继关系，CDT 层负责 parent 和 original 关系，最后管理器层再把这些局部结论组合成整体 wf，它的证明主线大致可以概括为，执行之后关系成立，cte_t 解释槽位的局部后状态，mdb 恢复局部图关系，cdt 恢复派生状态，最后再由管理器层把 wf 收回来，move 和 swap 也延续了同一套骨架，只是它们还会分别多出源槽清空、目标槽继承、双向映射，以及更复杂的邻接关系等证明义务。

5.4 删除传播操作验证实现

删除传播路径在实现组织上和 insert、move、swap 这些操作不太一样，它并不是把所有恢复义务都压到一个一次性的单一后状态里面，而是把删除之后的语义保持，分成几个可以接上的阶段来处理，包括单步删除、传播删除、zombie 收缩，以及 revoke 收尾等过程，这种差别首先表现在实现文件和证明组织方式上。

具体地，`set_empty` 负责建立空槽化、局部链路修补和基础 `parent` 语义保持；`delete_one` 负责把单步删除后的安全性、可接纳条件和管理器层 `wf` 保持连接起来；`delete_all` 负责组织“已处理区域”与“待处理区域”之间的阶段关系；`reduce_zombie` 负责 `zombie slot number` 更新及其分支行为；`revoke` 则给出对子派生链反复删除后的关键后置条件。合起来看，删除传播路径已经形成覆盖阶段化契约、部分正确性和管理器层 `wf` 保持的机械化结果，其完成口径与第 5.3 节中的 `insert` 线保持一致。

与局部更新类似，删除传播路径的工程结构也需要单独呈现。图 5-4 给出了删除相关实现文件和证明文件在仓库中的分布情况。

```

91
92 pub proof fn lemma_delete_preserves_manager_semantics_wf(
93     old_mgr: CSpaceManager,
94     new_mgr: CSpaceManager,
95     slot: SlotPtr,
96 )
97     requires
98         old_mgr.wf(),
99         new_mgr.local_structural_wf(),
100         new_mgr.slot_perms_wf(),
101         new_mgr.slot_dom_nonzero_wf(),
102         crate::cspace::cdt::proof::structural_wf_on(new_mgr.cdt@),
103         old_mgr.slot_dom().contains(slot),
104         !old_mgr.slot_is_empty(slot),
105         old_mgr.delete_reply_arch_orphan_safe(slot),
106         old_mgr.delete_old_next_incoming_edges_admissible(slot),
107         old_mgr.cte_delete_rel(&new_mgr, slot),
108     ensures
109         new_mgr.wf(),
110 {
111 >     lemma_delete_preserves_easy_wf_components( ...
115 >     );
116
117 >     lemma_delete_post_incoming_edges_admissible_from_old_next_admissible( ...
121 >     );
122
123 >     lemma_delete_preserves_all_incoming_edges_from_admissible( ...
127 >     );
128 }
129

```

图 5-4 删除传播实现与证明文件对应截图

从证明骨架来看删除线和插入线并不是两套完全分开的做法，`insert` 是通过一次局部更新直接得到一个单一后状态，然后再交给 `cte_t`、`mdb`、`cdt` 和管理器层去组合，删除传播则是把同样需要恢复的责任，拆成几个阶段性的后状态再分别通过空槽化、MD B remove、CDT 协同，以及管理器层契约来接住这些部分，也就是说二者其实共用的是同一套分层恢复逻辑，主要差别在于删除线还需要明确记录已经处理和还没有处理的区域，以及不同生命周期阶段的变化。

与这些删除操作并行的，还有 `cspace/mdb/table/remove.rs` 中的 `remove_node` 一类图原语。它负责 `forward chain`、`local symmetry` 和 `nonlive detached` 等结构恢复，但并不直接承担 `finalise` 或对象生命周期语义。这样的分工说明，删除传播的机械化结果并不是依赖单一函数黑盒给出，而是由空槽化、删除传播、`zombie` 收缩和 `MDB remove` 等几个层次协同形成。

6 验证结果、分析与讨论

6.1 评估目标、实验设置与指标

本章从四个角度给出评估：静态规模、`l4v` 规模参照、与证明擦除相关的运行时形态，以及构建与运行观察。

统计对象以 `sel4_cspace` 的 `Cspace` 相关实现为主，行数按非空非注释行统计；`proof fn`、`spec fn`、`#[verifier::external_body]` 和 `uninterp spec fn` 通过文本检索获得，用于观察证明产物分布与可信边界规模。文件数和核心实现行数聚焦 `Cspace` 主体实现，而 `proof fn`、`spec fn`、`external_body` 与 `uninterp spec fn` 的检索范围覆盖相关源码目录，因此两类数字不直接作比例比较。

6.2 代码与证明规模统计

本节统计覆盖 `sel4_cspace` 中与 `Cspace` 相关的实现，并同时记录 `proof fn`、`spec fn`、`#[verifier::external_body]` 和 `uninterp spec fn` 的出现次数。其作用在于说明实现规模与证明产物分布，而不替代 `Verus` 机械检查日志。

表 6-1 `Cspace` 核心层规模统计

统计项	值
<code>Cspace</code> 核心实现 Rust 文件数	43
<code>Cspace</code> 核心实现非空非注释行数	15802
<code>proof fn</code> 静态检索数	191
<code>spec fn</code> 静态检索数	235
普通与证明相关函数总检索数	521
<code>#[verifier::external_body]</code> 静态检索数	135

按文件职责把这些桥接条目归类到表 6-2 中。

表 6-2 可信边界分布与对主结论的影响

边界类别	数量（条目数）	对主结论的影响
运行时表示与观察桥接	123	若出现问题，首先影响位级/视图级解释，但不直接把高层 CSpace 语义整体黑盒化
槽位局部语义桥接	12	影响 cte_t 层的 derive/children/final 等局部前提，因此关系到局部更新与删除证明的输入强度
管理器组合与跨层桥接	12	直接影响管理器层 wf 收口与跨层组合，是本文主结论最应继续收缩的边界部分
体系结构与删除收尾桥接	10	主要限制 delete/finalise 生命周期收尾及体系结构相关外推，不改变管理器核心层已成立结论

从路径分布看，第一类主要集中在 capability/raw.rs、kernel_api/raw.rs、deps/raw.rs、cspace/resolve/raw.rs 和 cspace/cte/raw.rs 等原始表示桥接位置；其余三类则主要分布在 cspace/cte/*、cspace/manager/*、capability/zombie.rs 以及体系结构相关模块中。这也说明本文当前最大的边界数量并不位于高层 CSpace 语义本身，而主要位于表示提升与局部桥接处。

6.3 与 14v 的规模参照

本文选取 14v 中与 CSpace 最直接相关的能力空间抽象规范和不变量证明材料作为参照。

表 6-2 与 14v CSpace 相关材料的规模参照

材料	行数
本文 CSpace 核心实现行数	15802
14v 能力空间抽象规范	853
14v 能力空间不变量证明	4743

6.4 与证明擦除相关的运行时形态分析

除了规模之外，还需要考察在 Verus 中引入规格、幽灵状态和证明函数之后，运行

时代码是否仍保持与原实现相近的结构。

从正文关心的高层现象看，兼容入口层已经显著变薄，运行时主体逻辑主要被重组到分层模块之中。

表 6-3 运行时结构保持性的观察指标表

指标	结果
幽灵保留开关静态检索数	126
兼容入口行数	43
旧版参考实现对应入口行数	537
入口层结构对比结果	26 插入，520 删除

6.5 构建、验证与 sel4test 运行结果

本节给出 Verus 验证结果和 spike 平台上的运行观察。CSPACE 相关工件的全包验证得到 333 verified, 0 errors；此外，spike 平台成功完成 sel4test 镜像构建、启动与测试框架执行，最终汇总输出为 Test suite passed. 112 tests passed. 51 tests disabled 。

首先，图 6-2 可展示 resolve_address_bits 的函数级验证结果。该函数本身就是第 5.2 节的代表案例，因此直接展示其验证命令和结果，比展示整个 resolve 模块更直观。

```
root@rel4_dev_env:/workspace/rel4_kernel# cargo xtask verify --package sel4_cspace --features '' -- --verify-only-
module cspace::resolve::exec --verify-function resolve_address_bits

warning: `sel4_common` (lib) generated 6 warnings
Checking sel4_cspace v0.1.0 (/workspace/rel4_kernel/sel4_cspace)
note: verifying module cspace::resolve::exec (selected functions)

verification results:: 2 verified, 0 errors (partial verification with `--verify-*`)
Finished `dev` profile [unoptimized + debuginfo] target(s) in 4.57s
root@rel4_dev_env:/workspace/rel4_kernel#
```

图 6-2 resolve_address_bits 函数验证结果截图

其次，图 6-3 可展示 CSpaceManager::cte_insert 的函数级验证结果。cte_insert 是局部更新线最典型、的一个函数。

```
root@rel4_dev_env:/workspace/rel4_kernel# cargo xtask verify --package sel4_cspace --features '' -- --verify-only-
module cspace::manager::impl_insert --verify-function CSpaceManager::cte_insert

warning: `sel4_common` (lib) generated 6 warnings
Checking sel4_cspace v0.1.0 (/workspace/rel4_kernel/sel4_cspace)
note: verifying module cspace::manager::impl_insert (selected functions)

verification results:: 1 verified, 0 errors (partial verification with `--verify-*`)
Finished `dev` profile [unoptimized + debuginfo] target(s) in 4.16s
root@rel4_dev_env:/workspace/rel4_kernel#
```

图 6-3 CSpaceManager::cte_insert 函数验证结果截图

再次，图 6-4 可展示删除传播代表函数的验证结果。

```

root@rel4_dev_env:/workspace/rel4_kernel# cargo xtask verify --package sel4_cspace --features '' -- --verify-only-
module cspace::manager::impl_delete::delete_slot

warning: `sel4_common` (lib) generated 6 warnings
Checking sel4_cspace v0.1.0 (/workspace/rel4_kernel/sel4_cspace)
note: verifying module cspace::manager::impl_delete::delete_slot

verification results: 5 verified, 0 errors
Finished `dev` profile [unoptimized + debuginfo] target(s) in 4.57s
root@rel4_dev_env:/workspace/rel4_kernel#

root@rel4_dev_env:/workspace/rel4_kernel# cargo xtask verify --package sel4_cspace --features '' -- --verify-only-
module cspace::manager::impl_delete::empty_slot

warning: `sel4_common` (lib) generated 6 warnings
Checking sel4_cspace v0.1.0 (/workspace/rel4_kernel/sel4_cspace)
note: verifying module cspace::manager::impl_delete::empty_slot

verification results: 2 verified, 0 errors
Finished `dev` profile [unoptimized + debuginfo] target(s) in 3.91s
root@rel4_dev_env:/workspace/rel4_kernel#

root@rel4_dev_env:/workspace/rel4_kernel# cargo xtask verify --package sel4_cspace --features '' -- --verify-only-
module cspace::manager::impl_delete::reduce_zombie

warning: `sel4_common` (lib) generated 6 warnings
Checking sel4_cspace v0.1.0 (/workspace/rel4_kernel/sel4_cspace)
note: verifying module cspace::manager::impl_delete::reduce_zombie

verification results: 5 verified, 0 errors
Finished `dev` profile [unoptimized + debuginfo] target(s) in 3.71s
root@rel4_dev_env:/workspace/rel4_kernel#

```

图 6-4 删除传播代表函数验证结果截图

在给出局部模块证据之后，再展示全包验证结果会更有说服力。图 6-5 用于呈现 CSpace 相关工作件整体通过 Verus 检查。

```

root@rel4_dev_env:/workspace/rel4_kernel# cargo xtask verify --package sel4_cspace --features ''
size in C
= help: use `repr($int_ty)` instead to explicitly set the size of this enum
= note: `#[warn(repr_c_enums_larger_than_int)]` (part of `#[warn(future_incompatible)]`) on by default

warning: `sel4_common` (lib) generated 6 warnings
Checking sel4_cspace v0.1.0 (/workspace/rel4_kernel/sel4_cspace)
verification results: 333 verified, 0 errors
Finished `dev` profile [unoptimized + debuginfo] target(s) in 16.83s
root@rel4_dev_env:/workspace/rel4_kernel#

```

图 6-5 Verus 全包验证结果截图

除机械化验证之外，图 6-6 还给出了 sel4test 的汇总输出，用作实现可运行性的外部旁证。

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  xtask/target/x86_64-unknown-linux-gnu/release/xta

Starting test 110: VSPACE0006
Running test VSPACE0006 (Test touching all available ASID pools)
Test VSPACE0006 passed
Starting test 112: Test all tests ran
Test suite passed. 112 tests passed. 51 tests disabled.
All is well in the universe

Test end =====

```

图 6-6 kernel/sel4test 运行结果截图

6.6 结论边界与接口与系统级扩展

本文把结论边界限定在管理器核心层，并不是随便缩小研究范围，而是因为这一层同时有三个比较重要的特点，对下来说，它已经接住了槽位对象、MDB 图和 CDT 派生这三类关键语义责任，对上来说，它又可以作为接口层和系统调用层继续继承的语义基础，对横向的不同操作来说，它还能够通过统一的 wf 组合点，让证明骨架被重复使用，正是因为有这些特点，resolve、局部更新和删除传播这三类形态不一样的操作，才可以放在同一个结论口径下讨论，可信边界也可以被定位到更具体的桥接环节，而不是笼统停留在“管理器内部细节”这样的说法上。

如果要把本文的结论继续外推到更高层接口，关键并不是把核心操作重新证明一遍，而是需要补上从 CSpaceManager 到对外接口之间的契约继承关系，本文先在管理器核心层把证明责任收拢起来，是因为这一层的前提比较稳定，不变量也更集中，同时也更容易让局部更新和删除路径使用同一套证明骨架，与之对应的代价是，接口级语义和长期运行时一致性，还需要在后续工作中单独接上，进一步扩展时，至少还需要完成对外封装层和管理器层之间的契约对齐，说明面向 kernel 的封装入口语义，以及证明长期管理器视角和真实运行时 CSpace 状态之间的一致性。

6.7 研究局限和后续展望

本文的研究局限主要体现在三个方面，一是表示到语义之间的桥接仍然有所保留，二是 resolve_address_bits 仍然依赖底层读取和异常表示之间的桥接，三是对外封装层、面向 kernel 的封装层，以及长期管理器状态连接，目前还没有完全验证。后续工作仍然应该围绕对外封装层和管理器层之间的契约衔接，更多能力类型的纳入，桥接边界的继续收缩以及更完整的系统级语义连接来展开。

结束语

本文围绕 reL4 中的 sel4_cspace 模块，研究并完成了能力空间核心层在 Rust/Verus 环境下的形式化建模与主体验证工作。本文围绕 cap、mdb_node、cte_t、mdb、cdt 与 CSpaceManager 六层建立统一抽象状态模型，设计覆盖内容语义、链接语义、派生语义、untyped 约束与 zombie 条件的不变量体系，并将 CSpace 证明过程概括为“运行时最终后状态建立、分层局部后状态归约、管理器层分层组合恢复”的证明组织方法。在这一框架下，本文已经完成对 resolve_address_bits、cte_insert、insert_new_cap、cte_move、cte_swap 以及删除核心路径的规约设计、证明组织与主体实现，由此覆盖查询、解析、局部更新与删除传播等能力空间核心层行为。

从理论与工程意义上看，本工作一方面延续了 seL4/l4v 传统中能力空间语义严谨化的研究方向，另一方面说明 Verus 具备承载 Rust 系统软件复杂子系统验证的能力。

总体而言，本文已经为 reL4 CSpace 管理器核心层建立了可继续扩展的形式化验证基础，也为更广义的 Rust 微内核验证提供了一个可复用的能力系统验证样例。

致 谢

在本课题完成过程中，首先要感谢指导教师在选题、技术路线和论文写作方面给予的持续指导。能力系统、微内核验证与 Verus 工具链都具有较高门槛，许多关键判断都离不开老师在研究方向和论文规范上的帮助。

同时，感谢相关开源社区和前人研究成果提供的坚实基础。seL4、l4v 与 Verus 生态的研究，使本文能够站在已有高水平工作之上继续推进，也让在 Rust 环境中讨论内核能力空间验证成为现实可行的事情。

最后，感谢在毕业设计阶段给予帮助和支持的同学、朋友与家人。正是这些鼓励与协作，使得本论文能够从最初的验证设想逐步发展为一份具有完整结构的研究论文。

参考文献

- [1] 徐家乐, 王淑灵, 李黎明, 詹博华, 吕毅, 代艺博, 崔舍承, 吴鹏, 谭宇, 张学军, 詹乃军. 操作系统内核权能访问控制的形式验证[J]. 软件学报, 2025, 36(8): 3570-3586.
- [2] 钱汉伟, 王承毅. 操作系统形式化验证综述[J]. 河海大学学报(自然科学版), 2021, 49(5): 473-481.
- [3] 钱振江, 刘苇, 黄皓. 操作系统形式化设计与验证综述[J]. 计算机工程, 2012, 38(11): 234-238.
- [4] Klein G, Elphinstone K, Heiser G, 等. seL4: Formal Verification of an OS Kernel[A]. Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles[C]. New York: ACM, 2009: 207-220.
- [5] Klein G, Andronick J, Elphinstone K, 等. Comprehensive Formal Verification of an OS Microkernel[J]. ACM Transactions on Computer Systems, 2014, 32(1): 2:1-2:70.
- [6] Elphinstone K, Heiser G. From L3 to seL4: What Have We Learnt in 20 Years of L4 Microkernels?[A]. Proceedings of the 24th ACM Symposium on Operating Systems Principles[C]. New York: ACM, 2013: 133-150.
- [7] Sewell T, Winwood S, Gammie P, 等. seL4 Enforces Integrity[A]. Interactive Theorem Proving[C]. Berlin: Springer, 2011: 325-340.
- [8] Sewell T A L, Myreen M O, Klein G. Translation Validation for a Verified OS Kernel[A]. Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation[C]. New York: ACM, 2013: 471-482.
- [9] seL4 Foundation. seL4 Manual (latest version)[EB/OL]. <https://sel4.com/Info/Docs/seL4-manual-latest.pdf>, 2026-03-31/2026-05-11.
- [10] Jung R, Jourdan J H, Krebbers R, 等. RustBelt: Securing the Foundations of the Rust Programming Language[J]. Proceedings of the ACM on Programming Languages, 2018, 2(POPL): 66:1-66:34.
- [11] Lattuada A, Hance T, Cho C, 等. Verus: Verifying Rust Programs using Linear Ghost Types[J]. Proceedings of the ACM on Programming Languages, 2023, 7(OOPSLA1): 85:1-85:30.
- [12] Lattuada A, Hance T, Bosamiya J, 等. Verus: A Practical Foundation for Systems Verification[A]. Proceedings of the ACM Symposium on Operating Systems Principles[C]. New York: ACM, 2024: 438-454.
- [13] Hance T. Verifying Concurrent Systems Code[D]. Pittsburgh: Carnegie Mellon University, 2024.

- [14] Hawblitzel C, Howell J, Kapritsos M, 等. IronFleet: Proving Practical Distributed Systems Correct[A]. Proceedings of the ACM Symposium on Operating Systems Principles[C]. New York: ACM, 2015: 1-17.
- [15] Hance T, Zhou Y, Lattuada A, 等. Sharding the State Machine: Automated Modular Reasoning for Complex Concurrent Systems[A]. Proceedings of the 17th USENIX Symposium on Operating Systems Design and Implementation[C]. Berkeley: USENIX Association, 2023: 911-929.
- [16] LeBlanc H, Lorch J R, Hawblitzel C, 等. PoWER Never Corrupts: Tool-Agnostic Verification of Crash Consistency and Corruption Detection[A]. Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation[C]. Berkeley: USENIX Association, 2025: 839-857.
- [17] Zhang J, Xu X, Zou Y, 等. CortenMM: Efficient Memory Management with Strong Correctness Guarantees[A]. Proceedings of the ACM Symposium on Operating Systems Principles[C]. New York: ACM, 2025: 1082-1098.
- [18] Brun M, Achermann R, Chajed T, 等. Beyond Isolation: OS Verification as a Foundation for Correct Applications[A]. Proceedings of the 19th Workshop on Hot Topics in Operating Systems[C]. New York: ACM, 2023: 158-165.
- [19] Chen X, Li Z, Mesicek L, 等. Atmosphere: Towards Practical Verified Kernels in Rust[A]. Proceedings of the 1st Workshop on Kernel Isolation, Safety and Verification[C]. New York: ACM, 2023: 9-17.
- [20] Chen X, Li Z, Zhang J, 等. Atmosphere: Practical Verified Kernels with Rust and Verus[A]. Proceedings of the ACM Symposium on Operating Systems Principles[C]. New York: ACM, 2025: 752-767.
- [21] Zhou Z, Anjali, Chen W, 等. VeriSMo: A Verified Security Module for Confidential VMs[A]. Proceedings of the USENIX Symposium on Operating Systems Design and Implementation[C]. Berkeley: USENIX Association, 2024: 599-614.
- [22] Hance T, Elbeheiry L, Matsushita Y, 等. VerusBelt: A Semantic Foundation for Verus's Proof-Oriented Extensions to the Rust Type System[J]. Proceedings of the ACM on Programming Languages, 2026, 10(PLDI): 247:1-247:25.
- [23] Verus Team. Verus Publications and Projects[EB/OL]. <https://verus-lang.github.io/verus/publications-and-projects/>, 2026-05-11/2026-05-11.